

---

MODULE *block\_range*

---

get-block-range wire mechanics specification.

The draft's bulk download primitive ("get-block-range") streams a block and its ancestors in DESCENDING order from an anchor hash, bounded by an exact count and an exact byte budget: the responder stops before the block that would push the total past *max\_bytes* — EXCEPT that it MUST be willing to deliver the first block regardless, or an anchor block larger than the budget could never be fetched. Over-delivery is a connection error of type FLOOD ("both bounds are exact, so an honest responder never exceeds them"), and any truncation — bound reached or voluntary early finish — is resumable: the next anchor is the *hashPrevBlock* of the last delivered block, re-requested from the same or a different peer.

The model: a requester assembles the chain  $N \dots 1$  by repeated anchored requests against two responders, with adversarially chosen block sizes, truncation points and responder choice. Blocks are heights; the parent of  $h$  is  $h - 1$ ; the anchor after a delivery of  $\dots, h$  is  $h - 1$ .

The requester's FLOOD arithmetic must mirror the responder's stopping rule exactly. The *BuggyFirstInBudget* switch makes the requester count the FIRST block of a response against the byte budget — the natural off-by-one — and *NoHonestFlood* is violated: an honest responder delivering an over-budget anchor block (as the draft requires it to) is disconnected as a flooder. *RangeAssembled* and *AcceptedIsSuffix* verify that the resumption arithmetic loses nothing and duplicates nothing under every truncation pattern and peer switch.

See `../documents/v2-modeling.md` for the full write-up.

EXTENDS *Naturals, Sequences, FiniteSets*

|                                    |                                                                        |
|------------------------------------|------------------------------------------------------------------------|
| CONSTANT <i>MaxBlock</i>           | the chain is $1 \dots \text{MaxBlock}$ ; the range wanted is all of it |
| CONSTANT <i>Peers</i>              | responders; each holds the full chain                                  |
| CONSTANT <i>MaxCount</i>           | count limit the requester puts in each request                         |
| CONSTANT <i>MaxBytes</i>           | byte budget the requester puts in each request                         |
| CONSTANT <i>Sizes</i>              | the set block sizes are drawn from                                     |
| CONSTANT <i>BuggyFirstInBudget</i> | TRUE counts the first delivered block against the budget               |

*Blocks*  $\triangleq 1 \dots \text{MaxBlock}$

VARIABLES

|                   |                                                                                                    |
|-------------------|----------------------------------------------------------------------------------------------------|
| <i>size</i> ,     | [ <i>Blocks</i> $\rightarrow$ <i>Sizes</i> ] serialized size of each block (fixed at <i>Init</i> ) |
| <i>anchor</i> ,   | next height to fetch, descending; 0 = range complete                                               |
| <i>accepted</i> , | heights delivered and accepted                                                                     |
| <i>req</i> ,      | outstanding request: [ active, peer, a ]                                                           |
| <i>resp</i> ,     | response in flight: [ active, blocks : descending seq of heights ]                                 |
| <i>flood</i>      | TRUE once the requester closed a responder with FLOOD                                              |

*vars*  $\triangleq \langle \text{size}, \text{anchor}, \text{accepted}, \text{req}, \text{resp}, \text{flood} \rangle$

*NoReq*  $\triangleq [\text{active} \mapsto \text{FALSE}, \text{peer} \mapsto \text{"none"}, a \mapsto 0]$

*NoResp*  $\triangleq [\text{active} \mapsto \text{FALSE}, \text{blocks} \mapsto \langle \rangle]$

*SeqSum*(*s*)  $\triangleq \text{LET } F[i \in 0 \dots \text{Len}(s)] \triangleq \text{IF } i = 0 \text{ THEN } 0 \text{ ELSE } F[i - 1] + \text{size}[s[i]] \text{ IN } F[\text{Len}(s)]$

---

*Init*  $\triangleq$

$\wedge size \in [Blocks \rightarrow Sizes]$   
 $\wedge anchor = MaxBlock$   
 $\wedge accepted = \{\}$   
 $\wedge req = NoReq$   
 $\wedge resp = NoResp$   
 $\wedge flood = FALSE$

The requester asks any peer for the next span: anchored at its current frontier, with its standard count and byte budget.

$SendRequest \triangleq$   
 $\exists p \in Peers :$   
 $\wedge \neg flood$   
 $\wedge anchor > 0$   
 $\wedge \neg req.active \wedge \neg resp.active$   
 $\wedge req' = [active \mapsto TRUE, peer \mapsto p, a \mapsto anchor]$   
 $\wedge UNCHANGED \langle size, anchor, accepted, resp, flood \rangle$

The responder streams blocks descending from the anchor. It stops at the count bound, before a block (other than the first) that would exceed the byte budget — the first block is delivered regardless — or earlier at will (truncation is resumable). It never over-delivers.

$Serve \triangleq$   
 $\wedge req.active$   
 $\wedge LET a \triangleq req.a$   
 $\quad \text{the longest prefix the bounds allow}$   
 $\quad Fits(k) \triangleq \wedge k \leq MaxCount$   
 $\quad \quad \wedge k \leq a$   
 $\quad \quad \wedge \vee k = 1$   
 $\quad \quad \quad \vee SeqSum([i \in 1 .. k \mapsto a - i + 1]) \leq MaxBytes$   
 $IN \quad \exists k \in \{j \in 1 .. a : Fits(j)\} :$   
 $\quad \wedge resp' = [active \mapsto TRUE, blocks \mapsto [i \in 1 .. k \mapsto a - i + 1]]$   
 $\quad \wedge req' = NoReq$   
 $\quad \wedge UNCHANGED \langle size, anchor, accepted, flood \rangle$

The requester validates the response. The anchor and parent links hold by construction here (heights); the FLOOD arithmetic is the part under test: over-count, or over-budget by a block after the first — or, under the buggy switch, by any block including the first.

$RecvResponse \triangleq$   
 $\wedge resp.active$   
 $\wedge LET bs \triangleq resp.blocks$   
 $\quad overCount \triangleq Len(bs) > MaxCount$   
 $\quad overBudget \triangleq \exists k \in 1 .. Len(bs) :$   
 $\quad \quad \wedge (k > 1 \vee BuggyFirstInBudget)$   
 $\quad \quad \wedge SeqSum(SubSeq(bs, 1, k)) > MaxBytes$   
 $IN \quad \vee \wedge overCount \vee overBudget$

$$\begin{aligned}
& \wedge flood' = \text{TRUE} \\
& \wedge resp' = NoResp \\
& \wedge \text{UNCHANGED } \langle size, anchor, accepted, req \rangle \\
\vee & \wedge \neg(overCount \vee overBudget) \\
& \wedge accepted' = accepted \cup \{bs[i] : i \in 1 \dots Len(bs)\} \\
& \wedge anchor' = bs[Len(bs)] - 1 \\
& \wedge resp' = NoResp \\
& \wedge \text{UNCHANGED } \langle size, req, flood \rangle
\end{aligned}$$

---


$$Next \triangleq SendRequest \vee Serve \vee RecvResponse$$

$$Spec \triangleq Init \wedge \Box[Next]_{vars} \wedge WF_{vars}(Next)$$


---

Liveness: the whole range is eventually assembled.

$$RangeAssembled \triangleq \Diamond(accepted = Blocks)$$


---

Safety invariants.

$$\begin{aligned}
TypeOK & \triangleq \\
& \wedge anchor \in 0 \dots MaxBlock \\
& \wedge accepted \subseteq Blocks \\
& \wedge flood \in \text{BOOLEAN}
\end{aligned}$$

The resumption arithmetic never skips and never repeats: what has been accepted is exactly the suffix of the chain above the current anchor.

$$AcceptedIsSuffix \triangleq accepted = \{h \in Blocks : h > anchor\}$$

An honest responder, following the draft's exact bounds and the first-block rule, is never closed as a flooder.

$$NoHonestFlood \triangleq \neg flood$$


---